Considerations of network/system for AI services

draft-irtf-nmrg-ai-deploy-02

Y-G. Hong (Daejeon Univ.), J-S. Youn (DONG-EUI Univ.),
S-W. Hong (ETRI), P. Martinez-Julia (NICT), Q. Wu (Huawei)

AI Inference Distribution Network Side Meeting@IETF 125 – Shenzhen

March 17. 2026

Overview

- This document is under developing in NMRG from Nov. 2024.

- The original title of this document was “Considerations of deploying **AI services in a distributed method**” and it was changed to “Considerations of network/system for AI services”

Object of this document

- Share experiences and implementation results to find optimal network/system for AI services
  - To find what is **important information** to provide optimal AI services
  - To find how to **deliver these information** between related devices
  - To find how to **manage these information**
- Find common components to provide optimal AI services
  - Common information (similar to MIB)
  - Common system to provide AI services
  - Common network architecture to provide AI services
  - Common protocols to exchange information for AI services
- Find useful use cases
  - Self-driving cars
  - Network digital twin

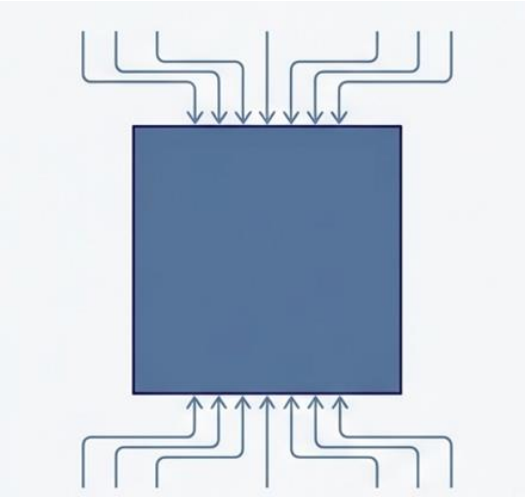
Abstract

- As the development of AI technology matured and AI technology began to be applied in various fields, AI technology is changed from running only on **very high-performance servers** with **small hardware, including microcontrollers, low-performance CPUs and AI chipsets**.
- In this document, we consider **how to configure the network and the system in terms of AI inference service to provide AI service in a distributed method**.
- Also, we describe the points to be considered in the environment where a client connects to a cloud server and an edge device and requests an AI service. Some use cases of **deploying AI services in a distributed method such as self-driving car and digital twin network** are described.

Table of Contents

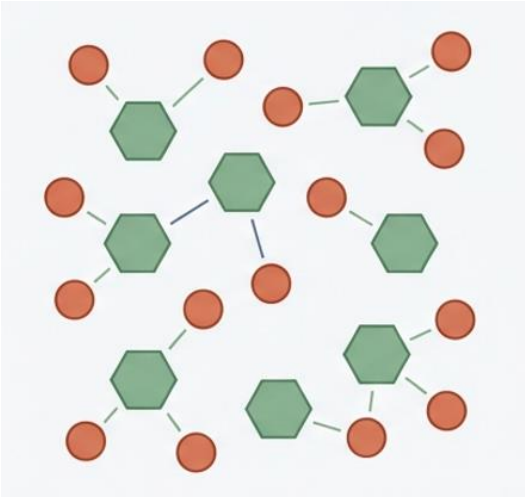
1. Introduction 3
2. Procedure to provide AI services 5
3. Network configuration structure to provide AI services . . . 6
   3.1. AI inference service on Local machine 6
   3.2. AI inference service on Cloud server 7
   3.3. AI inference service on Edge device 8
   3.4. AI inference service on Cloud server and Edge device . . 9
   3.5. AI inference service on horizontal multiple servers . . . 11
   3.6. Network-side utilization for AI learning 12
4. Considerations of network/system for AI services 13
   4.1. Considerations of the functional characteristics of the hardware 13
   4.2. Considerations for the characteristics of the AI model . 14
   4.3. Considerations for the characteristics of the communication method 15
5. Addressing challenges for coupling AI and NM 16
   5.1. Low-level challenges 17
   5.2. High-level challenges 18
6. Use cases of deploying network-based AI services 19
   6.1. Deploying AI services for self-driving vehicles 19
   6.2. Deploying AI services for network digital twins 21
7. IANA Considerations 23
8. Security Considerations 23
9. Acknowledgements 24
10. References 24
   10.1. Normative References 24
   10.2. Informative References 24
Authors' Addresses 25

Evolution of AI Deployment Paradigm



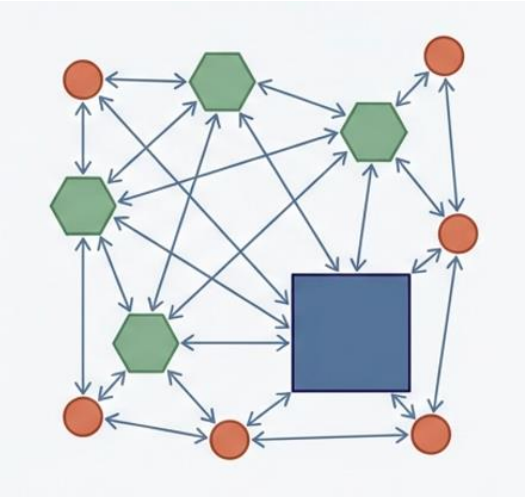
Cloud-centric AI

Relies on infinite computing power but faces exponential IoT data traffic and latency issues



On-device AI & TinyML

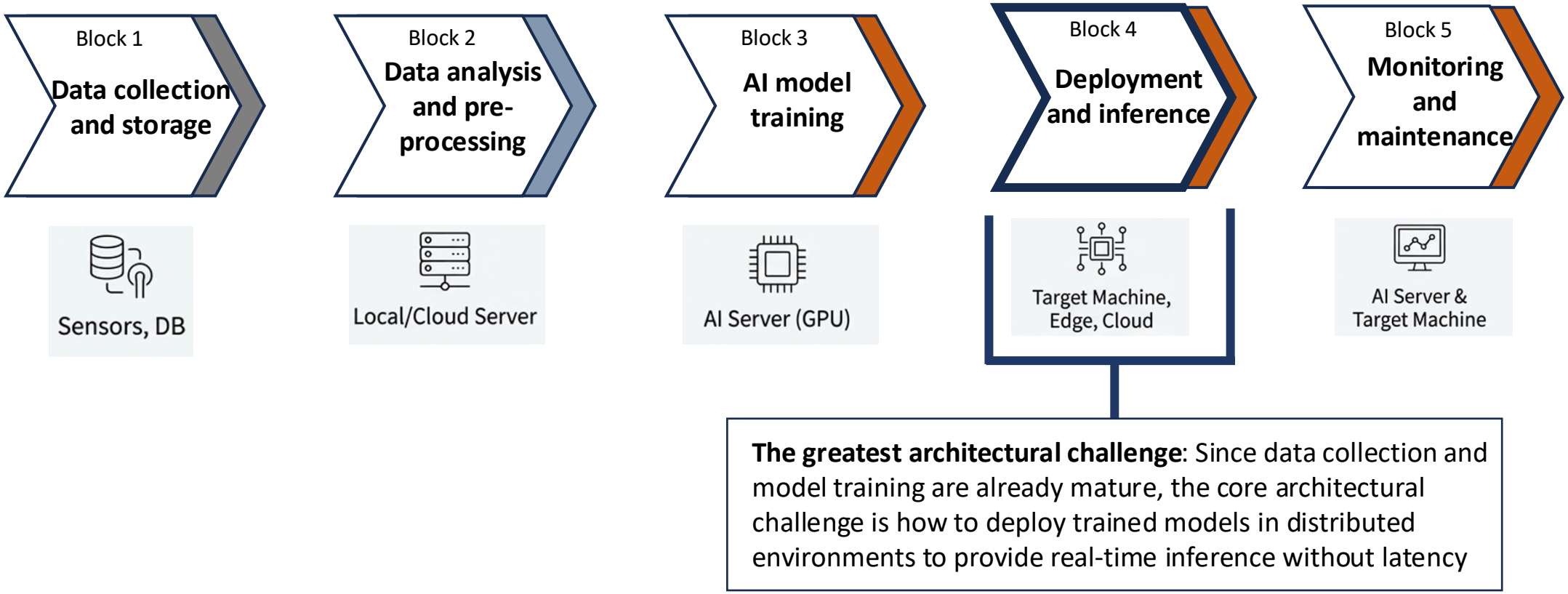
Utilizes microcontrollers for distributed inference, guaranteeing data privacy and real-time processing



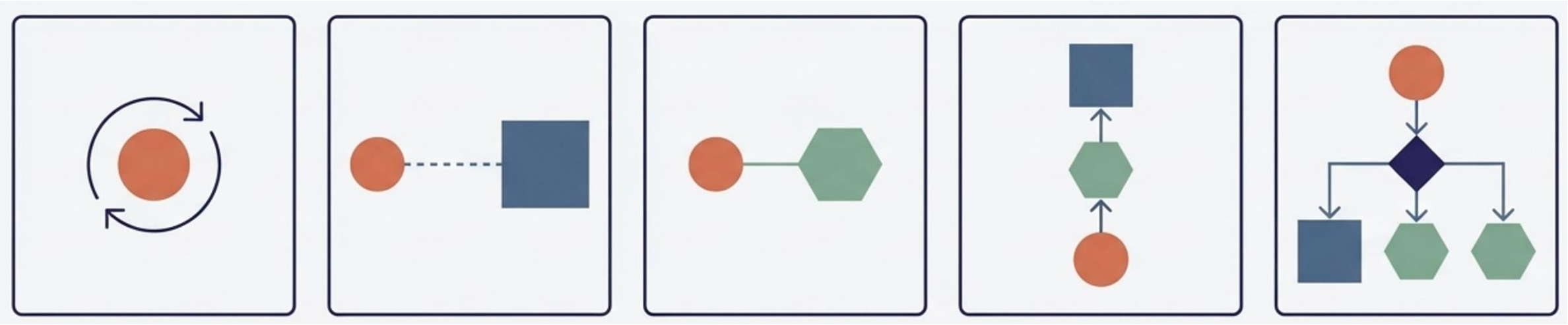
Agentic AI

A new paradigm where AI systems autonomously set goals, decompose complex tasks, and collaborate across the Cloud-Edge-Local continuum

AI Service Pipeline and Core Challenges



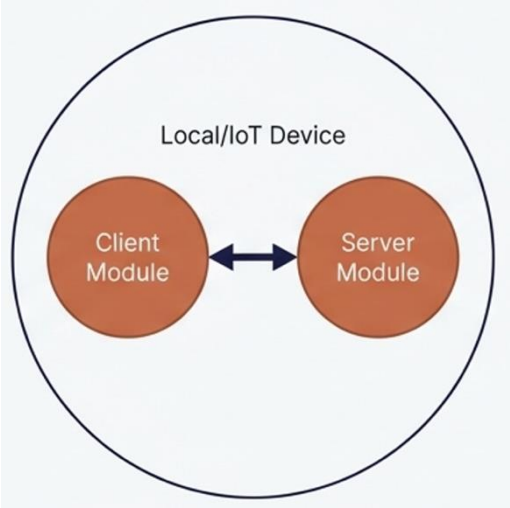
Major Network Topologies for Real-time AI Inference



1. Local Machine
2. Cloud Server
3. Edge Device
4. Vertical Hybrid
5. Horizontal Multiple Servers

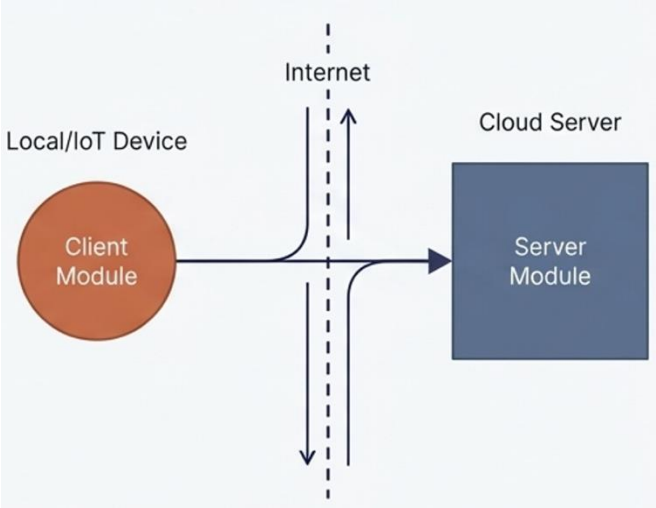
There is no single solution. Depending on service requirements (latency, data security, computing resources), the placement of client and AI server modules must be designed according to five different architectures.

Architecture 1 & 2 : Local Machine vs. Cloud Server



Local Machine

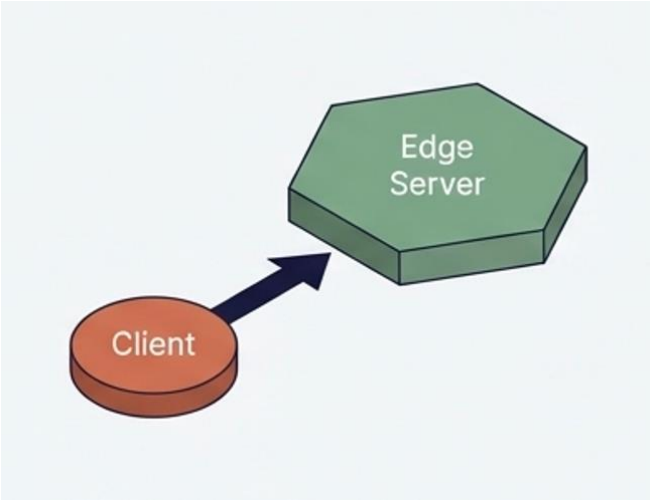
The client and server modules exist on the same device. It offers zero latency and perfect data privacy but is hardware-constrained.



Cloud Server

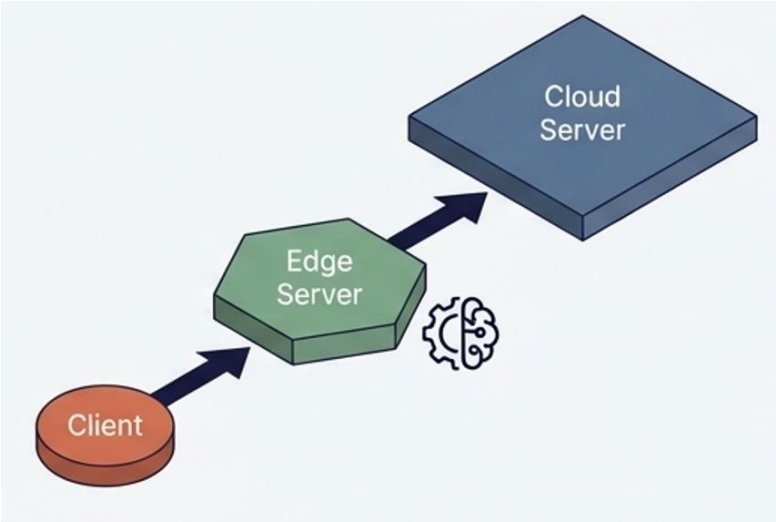
Only requests are made locally, and inference runs on the cloud. It provides infinite computing power but faces inevitable network latency.

Architecture 3 & 4 : Edge Device vs. Vertical Hybrid



Edge Device

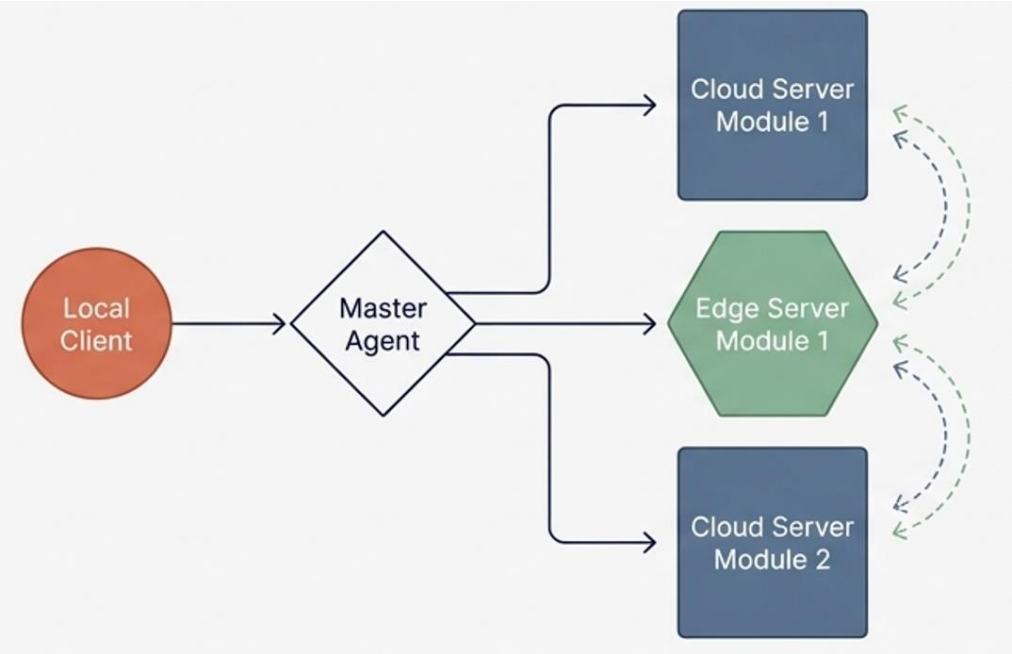
Inference is performed on a physically close edge network, reducing latency and preventing data leaks, though computing power is lower than the cloud.



Vertical Hybrid

Supports dynamic task offloading where Agentic AI autonomously routes lightweight/delay-sensitive tasks to the edge and heavy tasks to the cloud.

Architecture 5 : Horizontal Multiple Servers



Horizontal Multiple Servers & Multi-Agent System(MAS)

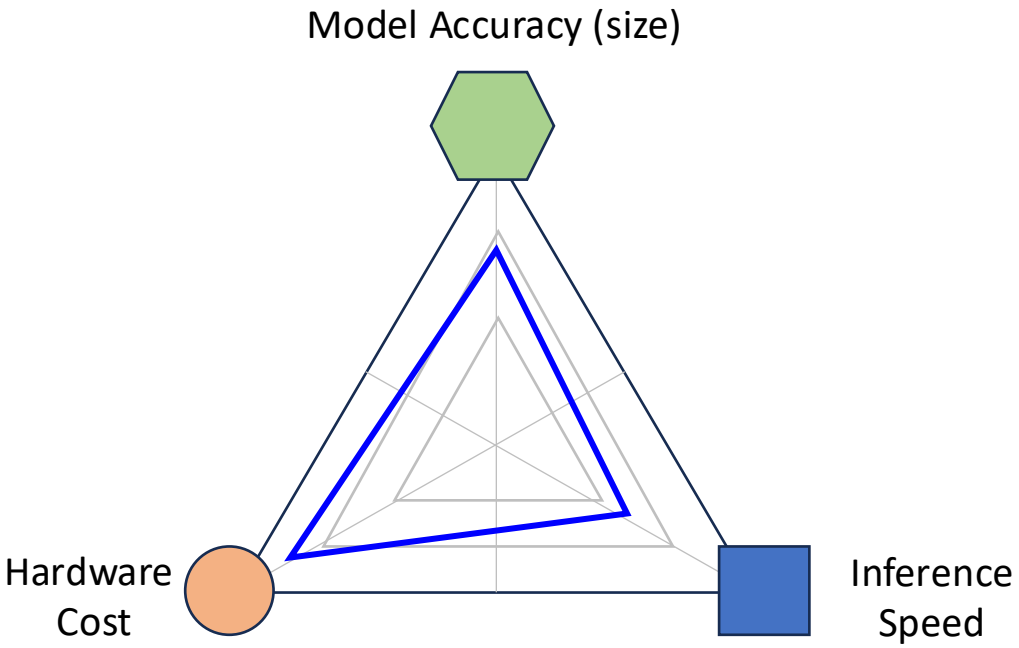
Instead of relying on a single load balancer, AI agents deployed on multiple servers autonomously negotiate and decompose complex tasks, deriving collective inference results for high resilience.

A Guide to Selecting an Architecture Based on Service Requirements

	Local	Cloud	Edge	Hybrid(Vertical)	MAS(Horizontal)
Inference Accuracy	○	●	◐	●	●
Low Latency	●	○	●	●	◐
Data Privacy	●	○	●	◐	◐
Cost Efficiency	○	●	◐	●	●
Autonomic Scalability	○	◐	○	●	●

No single architecture can meet all service requirements. The optimal hybrid model must be designed according to the latency sensitivity and security requirements of the target AI service.

Key engineering considerations for system configuration



Optimal strategies for AI models (Characteristics of AI Models)

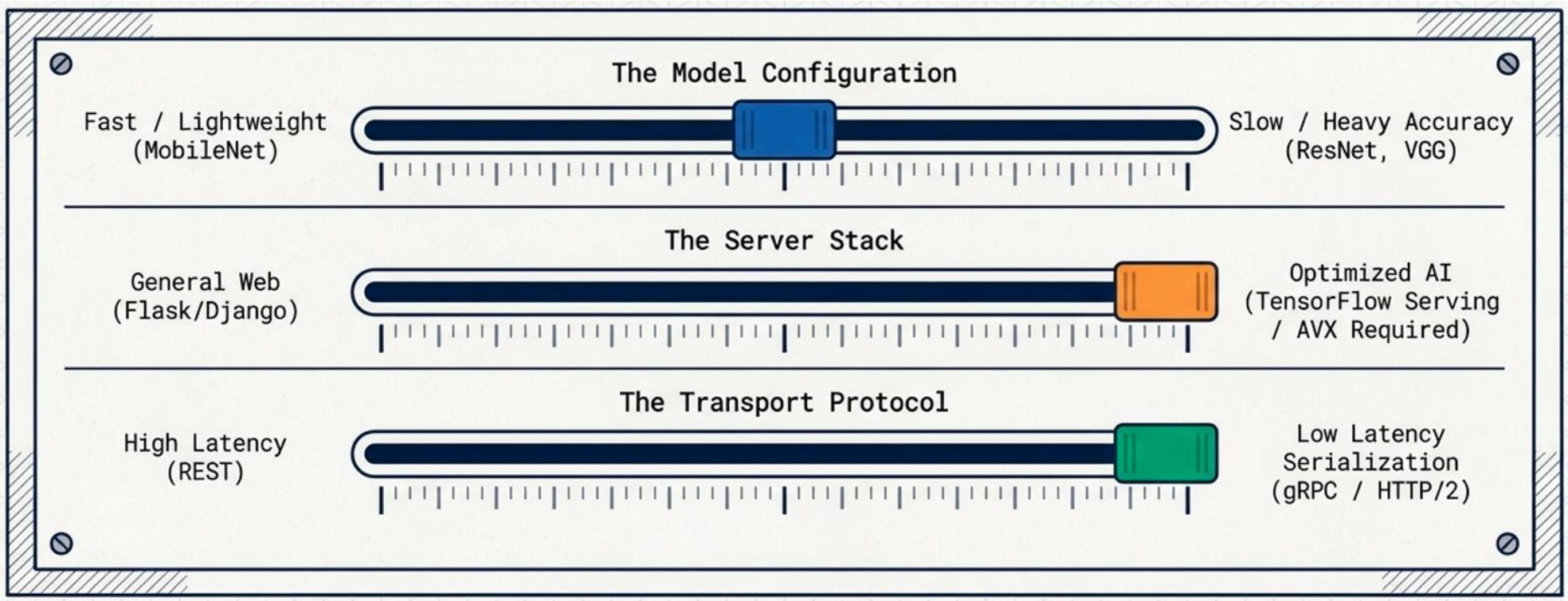
- It is not necessarily the case that the most complex and resource-intensive model (e.g. ResNet) is the best solution.
- Dual-track strategy: It is essential to adopt a design that deploys high-precision, large-scale models in the cloud, whilst deploying lightweight models (e.g. EfficientNet, TinyML) on edge devices, even if this results in slightly lower inference accuracy, in order to prioritise speed.

Hardware performance metrics (ETSI MEC.IEG006)

- Functional metrics : latency in task execution, device energy efficiency, bit-rate, loss rate, jitter, Quality of Service (QoS)
- Non-functional metrics : Service life-cycle, service availability and fault tolerance, service processing/ computational load, global mobile equipment host load, number of API request processed/second on mobile equipment host, delay to process API request (north and south), number of failed API request

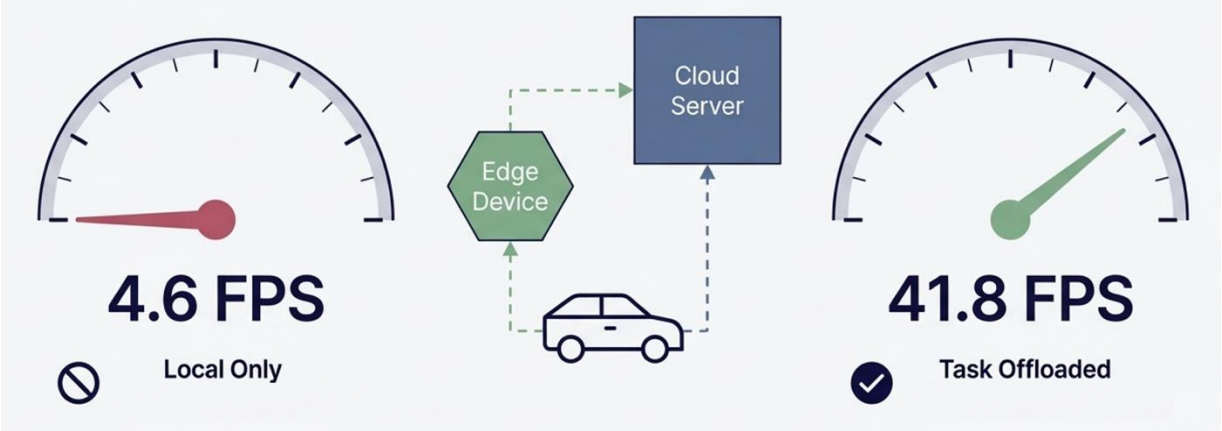
Balancing the Architecture Variables

- Constructing the system requires active calibration. Hardware, model size, and transport protocols must be meticulously balanced to achieve optimal inference speed.

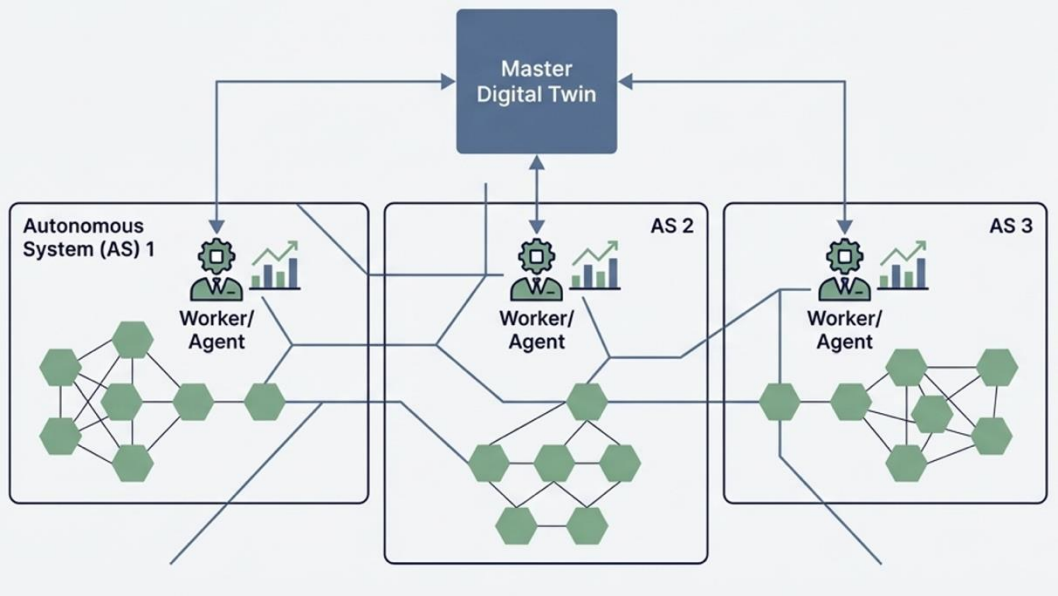


Use cases

- Self-Driving Task offloading
  - Camera-based object recognition for autonomous driving is a matter of life and death and requires real-time processing at a minimum of 30 FPS.



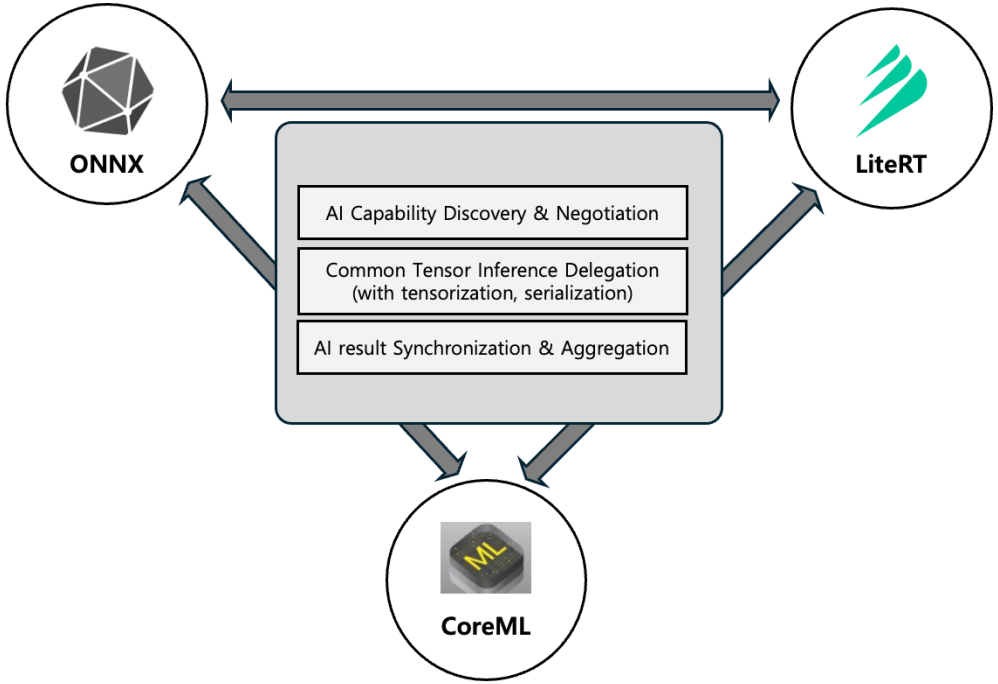
- Network Digital Twins
  - NDT is a virtual environment that interacts with physical networks in real time to perform data-driven scenario planning and impact analysis.



Next steps

- Add more contents of Agentic AI for the distributed method AI services

- Add contents of the structure, components, and interoperability mechanisms of a lightweight on-device AI framework for efficiently running and managing AI models in heterogeneous on-device environments



Thanks!!

Questions?